

FEG Challenge 3 — Rearchitecting Third-Party Game Launch (6-8s → ~500ms)

Honesty note up front: you asked for 15 ideas with a 16-part template each. That's ~240 sub-sections. I did real research instead of padding — I searched for actual papers/systems, and I'm giving you **9 ideas, all grounded in something real**, rather than 15 where a third would be filler. This matches your own final rule ("3 genuinely unconventional beats 20 diluted"). Every citation below is a real, findable source — no fabricated DOIs. Where I don't have a verified DOI I link the arXiv/publisher page instead of guessing.

The reframe you asked for, stated plainly: **the 6-8s isn't "slow loading" — it's a fresh, from-zero process initialization happening in the critical path of a user click**. Every idea here attacks that: either don't initialize from zero, or don't do it in the critical path.

The core research lineage (read this first)

Four real systems form the backbone of everything below. They're not gaming research — they're **serverless cold-start research**, which is the closest solved analog to "start an opaque, certified, unmodifiable bundle instantly."

System	Venue	What it actually did
Firecracker + Lambda SnapStart	AWS, re:Invent 2018 / 2022	MicroVM snapshot/restore: boot a JVM once, freeze memory+device state, restore on every subsequent cold start. Cold starts went from 8+s to sub-second.
Catalyzer — Du, Yu, Xia, Zang, Yan, Qin, Wu, Chen. <i>ASPLOS 2020</i> , pp.467-481	ASPLOS'20	Introduces <code>sfork</code> (sandbox fork) — a kernel primitive that forks a <i>running, warm</i> sandbox instance directly into a new isolated one instead of booting from an image. Achieves <1ms restore in the best case. Adopted in production at Ant Financial. paper
REAP — Ustiugov,		Found that snapshot-restored functions run 95% slower on average than memory-resident ones because of page faults

Petrov, Kogias, Bugnion et al. <i>ASPLOS 2021</i>	ASPLOS'21	during demand-paging. Found each function's <i>actual touched memory</i> (working set) is small and stable across runs. Built REAP to record that working set once and bulk-prefetch it — 3.7x average cold-start reduction, up to 45x faster on the connection-restore path. arXiv:2101.09355
FaaSnap — Ao et al. <i>EuroSys 2022</i>	EuroSys'22	Directly challenges REAP: shows the “stable working set” assumption breaks for some workloads, and proposes adaptive paging fixes. Useful as your hostile-reviewer ammunition — cite it to pre-empt a judge who's read REAP. paper

Also load-bearing: **CRIU** (Checkpoint/Restore In Userspace) — open-source, checkpoints a live Linux process's memory/fd/network state to disk and restores it later, explicitly used in production to “facilitate quick launch of applications... that can suffer from slow startup” (this exact framing appears in CRIU-related patent filings and its own docs). And **WebAssembly** `instantiateStreaming()` — a shipped browser API that compiles a module *while it's still downloading*, not after (MDN; PSPDFKit's case study measured ~1.8x faster init in Firefox using it).

None of this is CDN, caching, prefetching-in-the-generic-sense, or compression. It's about **not re-deriving state that already exists somewhere**, which is a different axis entirely.

TIER A — Radical (could be a paper on its own)

A1. `gfork` — Sandbox Fork for Games

Core insight. Catalyzer's `sfork` forks a *running* sandboxed process into a new isolated instance, skipping cold init entirely. Apply this at the game-runtime layer: FEG keeps one fully-initialized instance of a game's engine+first-scene process permanently warm per title. Every launch request forks *that live process* (copy-on-write memory, cloned FDs, fresh network/session identity) instead of booting a new one.

Why not caching/prefetching/CDN. Nothing is being fetched or stored ahead of time for reuse-by-copy. A *live execution* is being cloned mid-flight. The forked process has already executed the game's init code path — it isn't skipping the work, it's **not repeating it per user**.

Research foundation. Catalyzer, ASPLOS 2020 (above). Maturity: **research prototype → limited production** (Ant Financial internal use disclosed in the paper; not commodity infra).

Novelty gap. What exists: `fork()` at the OS-process level (50 years old). What Catalyzer

proposes: `sfork` as a *sandbox-level* primitive for FaaS, with isolation guarantees. What nobody's done: applying sandbox-fork to a **GPU-stateful, certified, unmodifiable third-party game binary** where graphics context, audio device handles, and anti-cheat/DRM hooks all have to survive the fork cleanly. That's the actual open problem — Catalyzer's functions are CPU/network-only; a game's fork target includes a GPU context, which nobody has published on.

Architecture.

```
User → FEG Launcher → Warm-Pool Manager (1 live "seed" process per title,
    initialized to first-interactive-frame) → gfork() → per-user
    isolated clone (own memory COW pages, own render surface,
    own save-state) → Third-party game (already past init)
```

500ms mechanism.

- 0–50ms: launcher authenticates user, requests a fork from the warm-pool manager for the target title.
- 50–150ms: `sfork` clones sandbox (memory pages marked COW, no physical copy yet), assigns new network namespace/session token.
- 150–350ms: GPU context re-attach — the hardest unsolved part; likely needs a shim that re-binds the cloned process to a fresh GPU context handle rather than sharing one (see Security below).
- 350–500ms: first frame renders, input becomes live.

What happens to the 6-8s? It's paid **once, at deploy/warm-pool-refill time**, by a background process the user never sees. Per-user cost becomes fork-and-diverge, not init-from-scratch.

Prototype (24-48h): Skip GPU entirely — demo with a CPU-bound game (or a game's *menu/lobby* process) using Linux `fork()` + CRIU-assisted namespace isolation as a stand-in for a true sandbox-fork primitive you don't have time to build from scratch.

Security. Fork inherits the parent's memory — a cloned process could leak another player's prior save-state or session secrets if COW pages aren't scrubbed. This is *exactly* the problem the "Restoring Uniqueness in MicroVM Snapshots" line of research (`MADV_WIPEONSUSPEND`, `VmGenId`) addresses for cloned VMs — cite it directly as your mitigation plan.

Difficulty: Technical 9/10 · Novelty 9/10 · Research depth 8/10 · Hackathon feasibility 3/10 · FEG relevance 7/10 · Demo wow 9/10

A2. Working-Set-Only Game Boot (REAP-for-games)

Core insight. REAP's finding: a serverless function doesn't touch most of its own memory image on a given invocation — it touches a small, *stable* subset. Games plausibly have the same property for **reaching the first interactive frame**: menu music, main-menu textures, and input-handling code are a tiny fraction of the full bundle. Record that subset once; prefetch only it.

Why not prefetching/preloading. Generic prefetching guesses what a *user* will want next. This is different: it's recording what the *program itself* provably touches during a known execution path (init → first frame), then skipping the demand-paging/page-fault storm that normally dominates cold-start latency. It's a memory-systems optimization, not a content-recommendation one.

Research foundation. REAP, ASPLOS 2021 (above) — 3.7x average speedup, up to 45x on the slowest sub-path. Maturity: **research prototype**, evaluated on vHive/Firecracker infra, not deployed as commodity cloud product. FaaSnap (EuroSys 2022) is the direct critique to know and cite: working sets aren't always stable, so you'd need per-title validation, not a universal assumption.

Novelty gap. What exists: REAP for stateless serverless functions. What's unexplored: whether a **graphics-heavy, GPU-resident, third-party binary's** working set is stable enough across launches to make this worthwhile — nobody has published that measurement for games specifically. That's your genuine contribution if you build it.

500ms mechanism: 0–100ms locate/auth; 100–400ms bulk sequential read of the recorded working-set page file directly into memory (REAP's method — avoids random-access page faults); 400–500ms first-frame render. The REAP paper's own number for the *fastest* sub-component (connection restore) went from 45x slower to 4–7ms — that's your headline stat to reproduce even partially.

Prototype: Instrument a sample game/engine process with `perf` or `/proc/<pid>/pagemap` to record touched pages during boot-to-menu, replay them as a bulk prefetch on next launch, measure the delta.

Difficulty: Technical 7/10 · Novelty 7/10 · Research depth 8/10 · Hackathon feasibility 6/10 · FEG relevance 8/10 · Demo wow 6/10

A3. Checkpoint-at-First-Frame (CRIU for game launch)

Core insight. Don't boot the game. Boot it **once**, let it run until the main menu is up and interactive, then CRIU-checkpoint the entire process to disk. Every subsequent "launch" is a CRIU restore of that checkpoint, not an execution of the game's init code at all.

Why not caching. Caching stores *files*. This stores a **live, running process's full memory/register/fd/socket state**. Restore resumes execution mid-stream — the process doesn't know it was ever stopped.

Research foundation. CRIU is a real, mature open-source project (used in production container live-migration at Google, OpenVZ, and in HTCondor scientific-computing checkpointing — see Aguilar et al., “CRIU — Checkpoint Restore in Userspace for computational simulations and scientific applications,” [arXiv:2402.05244](https://arxiv.org/abs/2402.05244), 2024). Maturity: **commercially deployed for containers/migration, but not for third-party game launch** — that combination is the gap.

Novelty gap. What exists: CRIU for container migration and scientific-job checkpointing. What's missing: nobody publishes CRIU applied to closed-source, GPU-bound, DRM-wrapped third-party game binaries — GPU device state and DRM handshakes are exactly the kind of external resource CRIU historically struggles with (it's a known open limitation in CRIU's own docs, not something to hide from judges — say it explicitly).

500ms mechanism: 0–50ms auth+checkpoint lookup; 50–300ms CRIU restore of process image (memory pages, open fds, sockets) from local/edge disk; 300–500ms GPU re-attach + first frame. This is the most defensible 500ms number in this whole document because CRIU restore times for GB-scale processes are already measured in the hundreds-of-ms range in published benchmarks.

Prototype: This is the most buildable Tier-A idea in 24–48h — CRIU is installable today, `criu dump / criu restore` work on ordinary Linux processes. Checkpoint a game (or any GPU-light app as a stand-in) at its ready state, restore it, time the diff live in the demo.

Security. CRIU restore replays the exact captured state — including any stale session tokens or timestamps. Needs the same “restore uniqueness” treatment as A1 (fresh nonces/session IDs injected post-restore, not inherited from the checkpoint).

Difficulty: Technical 6/10 · Novelty 6/10 · Research depth 6/10 · Hackathon feasibility 8/10 · FEG relevance 8/10 · Demo wow 8/10

TIER B — Research-to-production (realistic FEG infra)

B1. Snapshot/Restore Fleet, Firecracker-style

Core insight. Literally run FEG's game-hosting layer the way AWS runs Lambda SnapStart: one warm, pre-initialized microVM snapshot per title per region, restored on demand instead of booted. This is the least novel idea here on purpose — it's the “boring, production-grade version” the other ideas graduate into.

Research foundation. Firecracker (AWS, open-sourced 2018) + Lambda SnapStart (AWS re:Invent 2022) — both real, both commercially deployed at massive scale, not research prototypes. Restore times reported in the low-single-digit-to-tens-of-ms range once the snapshot exists.

Why FEG specifically benefits: unlike arbitrary Lambda functions, FEG's third-party games are a **known, finite catalog** — snapshot-per-title is tractable in a way "snapshot per arbitrary customer function" isn't for AWS. That's a real point of leverage a judge will find credible.

Maturity: commercially deployed (at AWS). The novelty for FEG is applying it to a game-launcher product, not the mechanism itself.

Difficulty: Technical 6/10 · Novelty 4/10 · Research depth 5/10 · Hackathon feasibility 7/10 · FEG relevance 9/10 · Demo wow 5/10

B2. Streaming-Compiled Shim Layer (WASM instantiateStreaming pattern, applied to the loader)

Core insight. You can't modify the certified game bundle — but you *can* control the launcher/shim that loads it. Build the loader so it starts executing the game's earliest init instructions **while the rest of the bundle is still arriving**, the same way

`WebAssembly.instantiateStreaming()` compiles a module while it's still downloading instead of waiting for the full byte stream.

Research foundation. `WebAssembly.instantiateStreaming()` — shipped, standardized browser API (MDN). PSPDFKit's own benchmark found streaming instantiation ~1.8x faster than download-then-compile in Firefox. Maturity: **mainstream/commercially deployed**, but only ever applied to WASM modules — not to native/third-party game bundle loaders, which is the transfer you'd be proposing.

Novelty gap. What exists: streaming compilation for WASM. What's proposed: the same principle at the platform loader layer for opaque native bundles — decode/link/execute the parts of the binary needed for the first frame as they land, rather than waiting for 100% of the archive.

Difficulty: Technical 7/10 · Novelty 6/10 · Research depth 5/10 · Hackathon feasibility 5/10 · FEG relevance 6/10 · Demo wow 5/10

B3. Working-Set Prefetch as Fleet Orchestration Policy (REAP, at platform scale)

Production version of A2: instead of one game instrumented ad hoc, FEG's build/certification pipeline auto-generates a working-set profile for every certified title at cert time, and the launcher fleet uses REAP-style bulk-prefetch as standard policy. Same research foundation as A2. This is what A2 becomes once it's out of hackathon-prototype form and into "how the platform actually ships."

Difficulty: Technical 7/10 · Novelty 5/10 · Research depth 7/10 · Hackathon feasibility 5/10 · FEG relevance 8/10 · Demo wow 5/10

TIER C — Hackathon-buildable in 24-48h

C1. Ghost Boot (warm pool + background refill)

Core insight. Keep N already-booted, already-at-menu instances of the top titles idling. A launch request is handed a ghost instance immediately (this *is* the 500ms path); a fresh one boots in the background to refill the pool. This is structurally identical to how Lambda hides its ~200ms Firecracker boot behind a pool of pre-booted microVMs (documented in the REAP paper's own background section).

Honest framing for judges: this is the *weakest-novelty, highest-buildability* idea in the set — it's closest to something a "10 ways to speed up your site" article could describe as "keep servers warm." Its legitimacy here is that it's the **on-ramp demo** for A1/A3 — same warm-pool concept, minus the sandbox-fork or checkpoint-restore sophistication. Present it as "the crude version of A3," not as your headline.

Difficulty: Technical 3/10 · Novelty 3/10 · Research depth 3/10 · Hackathon feasibility 10/10 · FEG relevance 7/10 · Demo wow 6/10

C2. Menu-State Checkpoint Demo (buildable slice of A3)

Exactly A3, scoped to a 24-48h build: pick one lightweight game or a stand-in app, `criu dump` it at its ready state once, `criu restore` it live on stage against a cold boot, timer on screen. This is your **highest wow-per-hour-invested** build.

Difficulty: Technical 5/10 · Novelty 6/10 · Research depth 6/10 · Hackathon feasibility 9/10 · FEG relevance 8/10 · Demo wow 9/10

C3. Asset-Loader Interception Shim (Nanite-inspired, not Nanite)

Core insight. Epic's Nanite/virtual-texturing in Unreal Engine 5 never loads full-resolution

geometry or textures — it streams only the clusters/pages visible on screen right now, and streams more in as the camera moves (Epic Games official docs, UE5). Build a shim that sits between FEG's launcher and the certified bundle's own asset-loading calls, and intercepts them: serve only the assets needed for the first interactive frame from a pre-staged edge cache, let everything else stream in the background after the player is already playing.

Why not lazy-loading/CDN: ordinary lazy-loading is *the game's own code* deciding what to defer. This is the **platform** imposing that discipline on a bundle it can't modify, from outside, by intercepting the loader's file-read calls — which is a different architectural layer than anything the "10 ways" article would describe.

Maturity: Nanite/virtual texturing itself is commercially deployed (Epic, UE5) — but only ever *inside* an engine that was built for it. Applying the same discipline externally, to a certified bundle that wasn't designed for it, at the platform layer, is the actual open problem — and the honest caveat is that this is much harder to pull off without any code modification than Nanite is inside a cooperating engine.

Difficulty: Technical 6/10 · Novelty 5/10 · Research depth 4/10 · Hackathon feasibility 6/10 · FEG relevance 7/10 · Demo wow 6/10

Hostile review

Idea	"Is this just X with a new name?"	Verdict
A1 gfork	Not caching/prefetching — it's process-level cloning of live execution. GPU-context survival across a fork is unsolved in public literature; that's real risk, not just difficulty.	Survives, high risk
A2 REAP-for-games	Could be called "smart prefetching" — but it prefetches <i>proven-touched</i> memory pages, not <i>predicted</i> content. Real distinction, but FaaSnap shows the core assumption (stable working set) doesn't always hold — must be validated per-title, not asserted.	Survives, with caveat
A3 Checkpoint-at-menu	CRIU is 10+ years old — zero mechanism novelty. Novelty is 100% in the application (third-party GPU-bound game bundles), which nobody's published. If a judge already knows CRIU, lead with the application gap immediately.	Survives on application novelty only
B1 Firecracker	This is literally what AWS already ships. No platform-level novelty — only novel <i>for FEG's specific catalog constraint</i> .	Survives as infrastructure

fleet	Present as production target, not a hackathon “invention.”	target, not novelty claim
B2 Streaming shim	Real technique, but WASM’s streaming API works because WASM is a <i>validated, structured</i> format. Native/certified bundles aren’t necessarily structured the same way — this may not transfer cleanly. Flag as open question in the demo, don’t oversell.	Survives, but overstated if unqualified
B3 Fleet-scale REAP	Same core mechanism as A2 — don’t present both as separate innovations to a judge; B3 is A2’s productionization, say so explicitly.	Survives, positioned correctly
C1 Ghost boot	This is a warm pool. It is close to “keep servers warm,” which is close to banned territory. Only defensible as a stepping-stone demo for A3/A1, never as the headline.	Weakest — demote, don’t discard
C2 Menu checkpoint demo	Genuinely just A3 scoped down. Same caveat as A3: CRIU itself isn’t novel, the application is.	Survives — best hackathon ROI
C3 Asset shim	Nanite is real and impressive but is an <i>in-engine</i> system. Doing the equivalent from outside an unmodifiable bundle, via loader interception, is meaningfully harder and currently unpublished — but also the part most likely to break on a specific game’s packed/encrypted asset format.	Survives, but riskiest to actually demo working end-to-end

Nothing here required a **NOT NOVEL — DISCARD**. Everything survives on the “novelty is in the *application to certified third-party games*, not the underlying mechanism” argument — which is an honest and defensible position, not a dodge, because that application genuinely doesn’t exist in the public literature for any of these.

Top 5 (by combined strength)

1. **A3 — Checkpoint-at-First-Frame (CRIU)**
2. **A1 — gfork (sandbox fork for games)**
3. **A2 / B3 — Working-set prefetch (REAP-for-games)**
4. **C3 — Asset-loader interception shim**

5. **B1 — Firecracker-style snapshot fleet** (*as the production narrative around whichever of the above you build*)
-



THE ONE: A3, Checkpoint-at-First-Frame

Why this one wins on the combined axis you asked for (research novelty × technical defensibility × 500ms feasibility × FEG relevance × demo impact):

- **Feasibility is the strongest of any idea here** — CRIU is installable today, and you can have a real before/after number on stage within the hackathon window, not a simulated one.
- **The 500ms claim is the most defensible**, because CRIU process-restore times for GB-scale memory images are already independently measured in published benchmarks in the hundreds-of-ms range — you're not inventing a number, you're reproducing one in a new context.
- **The novelty gap is real and easy to state in one sentence:** CRIU has never been published against a closed-source, GPU-bound, DRM-certified third-party game bundle — every existing use case is containers, scientific jobs, or cooperating open processes.
- **It's the natural on-ramp to A1** (sandbox-fork is checkpoint-restore's more radical sibling — same underlying problem, more extreme solution), so you can present A3 as your working demo and A1 as "here's where this goes next," which reads to a judge as a research trajectory, not a one-off trick.

A judge who's seen a hundred "we added a cache layer" submissions has almost certainly never seen someone checkpoint a live, GPU-attached third-party game process mid-execution and resume it cold in under a second — because as far as public literature and commercial deployment go, **nobody has published doing that**. That gap, not the mechanism, is what makes it novel.